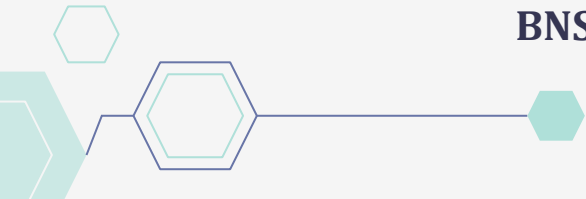


# Personal Portfolio

Ghina Hanifah Alamsyah

Uji Kompetensi Data Analis 2026  
BNSP





# Ghina Hanifah Alamsyah

Seseorang yang beraspirasi menjadi Data Analyst dan ahli strategi bisnis, serta memiliki ketertarikan pada bidang perbankan, otomotif, retail, dan juga fintech.



# Pendidikan

- Institut Teknologi Bandung | Bandung, Indonesia

Bachelor of Science in Physics | August 2021 – August 2025

**Relevant Coursework: Machine Learning, Statistical Analysis, Numerical Methods, Computational Physics**

# Pengalaman Bekerja

- PT Bank Syariah Indonesia Tbk | Jakarta, Indonesia  
Data Analyst Intern | December 2025 – June 2026
- ID/X Partners x Rakamin Academy | Jakarta, Indonesia (Remote)  
Data Science Project-Based Virtual Intern | January 2026 – February 2026

# Sertifikasi dan Skills

- Python Course — Progate | August 2021 – September 2021
- Data Science Specialization — MySkill | October 2025 – October 2025
- SQL Course — Mimo | October 2025 – October 2025
- Learn R — Codecademy | November 2025 – December 2025
- PyTorch and Neural Networks — Codecademy | December 2025 – January 2026
- Sertifikasi Kompetensi BNSP Data Analyst — DQLab | June 2026 – July 2026

## **Skills:**

- **Python**
- **SQL**
- **R**
- **Microsoft Excel**
- **Power BI**
- **Google Spreadsheet**
- **MySQL**
- **HeidiSQL**
- **Google Colab**
- **Visual Studio Code**

# Portofolio – Retail Store Sales Analytics

Bahasa pemrograman dan software yang digunakan: **SQL, MySQL, Python, dan Google Colab**

Dataset ini terdiri dari tiga tabel, yaitu customers.csv (1,200 row), products.csv (80 row), dan orders.csv (4,000 row).

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

|   | customer_id | gender | age | city       | signup_date | loyalty_member |
|---|-------------|--------|-----|------------|-------------|----------------|
| ▶ | C0001       | Male   | 19  | Liverpool  | 2023-10-17  | Yes            |
|   | C0002       | Male   | 65  | Manchester | 2022-08-17  | Yes            |
|   | C0003       | Female | 20  | London     | 2023-08-28  | Yes            |
|   | C0004       | Male   | 50  | London     | 2022-08-12  | Yes            |
|   | C0005       | Female | 32  | Nottingham | 2023-10-28  | No             |
|   | C0006       | Male   | 62  | Sheffield  | 2022-01-07  | No             |
|   | C0007       | Male   | 31  | Bristol    | 2022-10-12  | Yes            |
|   | C0008       | Female | 24  | Bristol    | 2022-04-05  | No             |
|   | C0009       | Female | 20  | Nottingham | 2023-09-11  | Yes            |
|   | C0010       | Male   | 53  | Liverpool  | 2023-01-23  | No             |
|   | C0011       | Male   | 63  | Manchester | 2023-08-15  | Yes            |
|   | C0012       | Male   | 36  | Manchester | 2023-11-09  | Yes            |
|   | C0013       | Female | 35  | Nottingham | 2022-04-14  | No             |

**customers.csv**

Result Grid | Filter Rows: | Export: |

|   | product_id | product_name      | category    | price  |
|---|------------|-------------------|-------------|--------|
| ▶ | P001       | Blender 1         | Beauty      | 272.76 |
|   | P002       | Perfume 2         | Home        | 132.09 |
|   | P003       | Water Bottle 3    | Home        | 58.58  |
|   | P004       | Smart Watch 4     | Home        | 220.82 |
|   | P005       | Perfume 5         | Home        | 27.81  |
|   | P006       | Vacuum Cleaner 6  | Beauty      | 59.03  |
|   | P007       | Laptop Stand 7    | Home        | 244.58 |
|   | P008       | Smart Watch 8     | Electronics | 93.18  |
|   | P009       | USB Cable 9       | Clothing    | 156.62 |
|   | P010       | Yoga Mat 10       | Home        | 48.44  |
|   | P011       | Yoga Mat 11       | Home        | 258.78 |
|   | P012       | Wireless Mouse 12 | Electronics | 185.11 |
|   | P013       | Perfume 13        | Clothing    | 201.16 |

**products.csv**

Result Grid | Filter Rows: | Export: | Wrap Cell Content

|   | order_id | customer_id | product_id | order_date | quantity | payment_method |
|---|----------|-------------|------------|------------|----------|----------------|
| ▶ | O00001   | C0692       | P006       | 2023-01-29 | 4        | Card           |
|   | O00002   | C1072       | P059       | 2023-04-19 | 5        | Card           |
|   | O00003   | C1187       | P030       | 2024-01-15 | 1        | Card           |
|   | O00004   | C0290       | P063       | 2024-01-12 | 1        | Cash           |
|   | O00005   | C0094       | P032       | 2024-04-09 | 2        | Card           |
|   | O00006   | C0103       | P040       | 2023-03-30 | 1        | Cash           |
|   | O00007   | C0350       | P076       | 2023-11-09 | 1        | Card           |
|   | O00008   | C1019       | P077       | 2023-02-28 | 5        | Card           |
|   | O00009   | C0416       | P055       | 2023-02-02 | 3        | Online         |
|   | O00010   | C0824       | P042       | 2023-12-14 | 1        | Online         |
|   | O00011   | C1118       | P018       | 2023-10-22 | 3        | Card           |
|   | O00012   | C0094       | P076       | 2024-01-27 | 2        | Cash           |
|   | O00013   | C0248       | P036       | 2023-02-02 | 2        | Cash           |

**orders.csv**



# Portofolio – Retail Store Sales Analytics

Adapun objektif/tujuan dari dilakukannya analisis data ini yaitu:

1. Mengetahui demografi customer/pelanggan yang berbelanja di perusahaan tersebut;
2. Menentukan statistik performa sales/penjualan di perusahaan tersebut dan mengimprovisasi performa penjualannya dengan menentukan strategi bisnis berdasarkan insight-insight yang telah diperoleh.

# Analisis Data Menggunakan SQL

1. Ada berapa banyak pelanggan/customers pada dataset?

Query:

```
SELECT COUNT(customer_id)
FROM customers;
```

Output:

|   | COUNT(customer_id) |
|---|--------------------|
| ▶ | 1200               |

2. Ada berapa banyak order/pesanan yang dibuat oleh pelanggan?

Query:

```
SELECT COUNT(order_id)
FROM orders;
```

Output:

|   | COUNT(order_id) |
|---|-----------------|
| ▶ | 4000            |



### 3. Top 5 kota mana saja yang memiliki jumlah pelanggan terbanyak?

Query:

```
SELECT city, COUNT(customer_id) AS total_customers
FROM customers
GROUP BY city
ORDER BY total_customers DESC
LIMIT 5;
```

Output:

|   | city       | total_customers |
|---|------------|-----------------|
| ▶ | Sheffield  | 163             |
|   | Nottingham | 160             |
|   | Birmingham | 159             |
|   | Bristol    | 156             |
|   | London     | 146             |

### 4. Top 5 produk terjual terbanyak?

Query:

```
SELECT products.product_id, products.product_name, SUM(orders.quantity) AS total_order
FROM products
INNER JOIN orders
ON products.product_id = orders.product_id
GROUP BY product_id, product_name
LIMIT 5;
```

Output:

|   | product_id | product_name     | total_order |
|---|------------|------------------|-------------|
| ▶ | P006       | Vacuum Cleaner 6 | 177         |
|   | P059       | Perfume 59       | 175         |
|   | P030       | Coffee Maker 30  | 165         |
|   | P063       | Jacket 63        | 155         |
|   | P032       | Keyboard 32      | 157         |

## 5. Total revenue/pendapatan yang dihasilkan?

Query:

```
SELECT SUM(products.price * orders.quantity) AS final_price
FROM products
INNER JOIN orders
ON products.product_id = orders.product_id;
```

Output:

|   | total_revenue      |
|---|--------------------|
| ▶ | 1644817.7099999872 |

## 6. Kategori produk mana yang menghasilkan pendapatan terbanyak?

Query:

```
SELECT products.category, SUM(products.price * orders.quantity) AS total_revenue
FROM products
INNER JOIN orders
ON products.product_id = orders.product_id
GROUP BY category
ORDER BY total_revenue DESC;
```

Output:

|   | category    | total_revenue      |
|---|-------------|--------------------|
| ▶ | Home        | 473002.140000001   |
|   | Sports      | 403963.88999999943 |
|   | Clothing    | 319942.70000000007 |
|   | Beauty      | 239438.96000000005 |
|   | Electronics | 208470.02000000014 |

## 7. Rata-rata jumlah order?

Query:

```
SELECT ROUND(AVG(quantity)) AS avg_order_quantity
FROM orders;
```

Output:

|   | avg_order_quantity |
|---|--------------------|
| ▶ | 3                  |

## 8. Kota yang menghasilkan pendapatan terbanyak?

Query:

```
SELECT customers.city, SUM(products.price * orders.quantity) AS total_revenue
FROM customers
INNER JOIN orders
ON customers.customer_id = orders.customer_id
INNER JOIN products
ON products.product_id = orders.product_id
GROUP BY city
ORDER BY total_revenue DESC;
```

Output:

|   | city       | total_revenue      |
|---|------------|--------------------|
| ▶ | Sheffield  | 233607.88000000015 |
|   | Birmingham | 224892.95000000003 |
|   | Nottingham | 213151.65999999992 |
|   | Bristol    | 202317.33          |
|   | Liverpool  | 197613.63000000001 |
|   | Leeds      | 194316.01000000018 |
|   | London     | 194041.93000000025 |
|   | Manchester | 184876.32000000003 |

## 9. Top 10 pelanggan dengan kontribusi pengeluaran terbanyak?

Query:

```
SELECT customers.customer_id, SUM(products.price * orders.quantity) AS total_spent
FROM customers
INNER JOIN orders
ON customers.customer_id = orders.customer_id
INNER JOIN products
ON products.product_id = orders.product_id
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 10;
```

Output:

|   | customer_id | total_spent        |
|---|-------------|--------------------|
| ▶ | C0299       | 5124.21            |
|   | C0094       | 5078.51            |
|   | C0588       | 4874.3             |
|   | C0826       | 4685.22            |
|   | C0334       | 4515.14            |
|   | C0291       | 4508.31            |
|   | C0765       | 4505.37            |
|   | C0157       | 4459.5499999999999 |
|   | C1167       | 4425.5300000000001 |
|   | C0674       | 4356.89            |

## 10. Bulan dengan penjualan tertinggi?

Query:

```
SELECT MONTHNAME(orders.order_date) AS month, SUM(orders.quantity * products.price) AS sales
FROM orders
INNER JOIN products
ON orders.product_id = products.product_id
GROUP BY month
ORDER BY sales DESC;
```

Output:

|   | month     | sales              |
|---|-----------|--------------------|
| ▶ | January   | 236205.87000000002 |
|   | February  | 209648.54999999996 |
|   | March     | 200317.80999999994 |
|   | April     | 194774.85000000012 |
|   | May       | 136058.51000000004 |
|   | June      | 107246.94000000006 |
|   | July      | 100780.97000000002 |
|   | August    | 97081.04           |
|   | October   | 96044.40000000005  |
|   | September | 93224.01999999999  |
|   | December  | 93131.93000000002  |
|   | November  | 80302.82000000007  |

## 11. Rata-rata pengeluaran tiap pelanggan?

Query:

```
SELECT AVG(table5.final_price) AS avg_spent
FROM(
    SELECT customers.customer_id, SUM(products.price * orders.quantity) AS final_price
    FROM customers
    INNER JOIN orders
    ON customers.customer_id = orders.customer_id
    INNER JOIN products
    ON products.product_id = orders.product_id
    GROUP BY customer_id
) AS table5;
```

Output:

|   | avg_spent          |
|---|--------------------|
| ▶ | 1431.5210704960846 |

## 12. Metode pembayaran yang paling umum digunakan?

Query:

```
SELECT payment_method, COUNT(payment_method) AS total_usage
FROM orders
GROUP BY payment_method
ORDER BY total_usage DESC;
```

Output:

|   | payment_method | total_usage |
|---|----------------|-------------|
| ► | Cash           | 1362        |
|   | Card           | 1345        |
|   | Online         | 1293        |

## 13. Kategori produk yang unitnya paling banyak terjual?

Query:

```
SELECT products.category, SUM(orders.quantity) AS total_unit_sold
FROM products
INNER JOIN orders
ON products.product_id = orders.product_id
GROUP BY category
ORDER BY total_unit_sold DESC;
```

Output:

|   | category    | total_unit_sold |
|---|-------------|-----------------|
| ► | Sports      | 3250            |
|   | Home        | 3074            |
|   | Clothing    | 2101            |
|   | Electronics | 1851            |
|   | Beauty      | 1623            |

Untuk dataset yang sama, namun menggunakan **Python**

## 1. Data Collecting

```
# Import the required libraries
import pandas as pd
import matplotlib.pyplot as plt

# Read .csv files
customers = pd.read_csv("/content/retail_sales/customers.csv")
orders = pd.read_csv("/content/retail_sales/orders.csv")
products = pd.read_csv("/content/retail_sales/products.csv")

# Join all the datasets together using inner join
merged_df = pd.merge(customers, orders, on='customer_id', how='inner')
merged_df = pd.merge(merged_df, products, on='product_id', how='inner')
merged_df.head()
```



## 2. Data Preparation

```
# Find total of rows and columns
merged_df.shape
```

```
(4000, 14)
```

```
# Find the dataframe's info
merged_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4000 entries, 0 to 3999
Data columns (total 14 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   customer_id     4000 non-null   object  
 1   gender          4000 non-null   object  
 2   age             4000 non-null   int64   
 3   city            4000 non-null   object  
 4   signup_date     4000 non-null   object  
 5   loyalty_member  4000 non-null   object  
 6   order_id        4000 non-null   object  
 7   product_id      4000 non-null   object  
 8   order_date      4000 non-null   object  
 9   quantity        4000 non-null   int64   
10   payment_method  4000 non-null   object  
11   product_name    4000 non-null   object  
12   category        4000 non-null   object  
13   price           4000 non-null   float64  
dtypes: float64(1), int64(2), object(11)
```

```
# Spot missing values
missing_values = merged_df.isnull().sum()
missing_values
```

```
0
customer_id  0
gender       0
age          0
city         0
signup_date  0
loyalty_member  0
order_id     0
product_id   0
order_date   0
quantity     0
payment_method  0
product_name  0
category     0
price        0
```

```
dtype: int64
```

```
# Check if there are duplicate values
duplicates = merged_df.duplicated().sum()
duplicates

np.int64(0)
```

```
# Describe the data
merged_df.describe()
```

|       | age         | quantity    | price       |
|-------|-------------|-------------|-------------|
| count | 4000.000000 | 4000.000000 | 4000.000000 |
| mean  | 41.535000   | 2.974750    | 139.516300  |
| std   | 13.627113   | 1.411599    | 86.734028   |
| min   | 18.000000   | 1.000000    | 11.070000   |
| 25%   | 30.000000   | 2.000000    | 59.030000   |
| 50%   | 41.000000   | 3.000000    | 120.140000  |
| 75%   | 53.000000   | 4.000000    | 213.020000  |
| max   | 65.000000   | 5.000000    | 289.840000  |

### 3. Data Analysis

```
# Find customers with the highest spend
data_per_customer = (merged_df.groupby('customer_id')
                     .agg({'order_id': 'nunique',
                          'quantity': 'sum',
                          'gmv': 'sum'})
                     .reset_index()
                     .rename(columns={'order_id': 'orders'}))

print(data_per_customer.sort_values(by='gmv', ascending=False))
```

|     | customer_id | orders | quantity | gmv     |
|-----|-------------|--------|----------|---------|
| 289 | C0299       | 8      | 28       | 5124.21 |
| 89  | C0094       | 8      | 24       | 5078.51 |
| 568 | C0588       | 9      | 27       | 4874.30 |
| 790 | C0826       | 7      | 25       | 4685.22 |
| 323 | C0334       | 8      | 30       | 4515.14 |
| ..  | ...         | ...    | ...      | ...     |
| 105 | C0111       | 1      | 1        | 41.44   |
| 414 | C0428       | 1      | 3        | 40.32   |
| 693 | C0724       | 1      | 1        | 36.66   |
| 181 | C0191       | 1      | 3        | 33.21   |
| 778 | C0813       | 1      | 3        | 33.21   |

[1149 rows x 4 columns]

```
# Find average spend per customers
data_per_customer = (merged_df.groupby('customer_id')
                     .agg({'order_id': 'nunique',
                          'quantity': 'sum',
                          'gmv': 'sum'})
                     .reset_index()
                     .rename(columns={'order_id': 'orders'}))

print(data_per_customer['gmv'].mean())

*** 1431.5210704960834
```

```
# Define age bins and labels
bins = [18, 25, 32, 40, 50, 60, 70]
labels = ['18-25', '26-32', '33-40', '41-50', '51-60', '61+']

# Create a new 'age_group' column based on the defined bins
merged_df['age_group'] = pd.cut(merged_df['age'], bins=bins, labels=labels, right=False)

# Display the first few rows with the new 'age_group' column
display(merged_df.head())
```

```
# Calculate the count of 'Male' and 'Female' for each age group
gender_age_group_counts = merged_df.groupby(['age_group', 'gender']).size().unstack(fill_value=0)

# Prepare data for pyramid plot: make male counts negative for left side of the plot
gender_age_group_counts['Male'] = -gender_age_group_counts['Male']

# Ensure the index (age_group) is in the desired order for plotting
gender_age_group_counts = gender_age_group_counts.reindex(labels)

display(gender_age_group_counts)
```

/tmp/ipykernel\_946/1471885278.py:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. To suppress this warning, please pass the observed parameter to the function.

|           | Female | Male |
|-----------|--------|------|
| age_group |        |      |
| 18-25     | 295    | -281 |
| 26-32     | 273    | -277 |
| 33-40     | 312    | -385 |
| 41-50     | 471    | -407 |
| 51-60     | 362    | -467 |
| 61+       | 219    | -251 |

Menambahkan kolom baru berupa label rentang umur untuk mempermudah visualisasi data

## 4. Visualisasi Data

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

fig.suptitle("Perbandingan Jumlah Pelanggan Laki-laki dan Perempuan\nBerdasarkan Kelompok Usia",
             x=0., y=1.0, fontsize=24, ha="left")

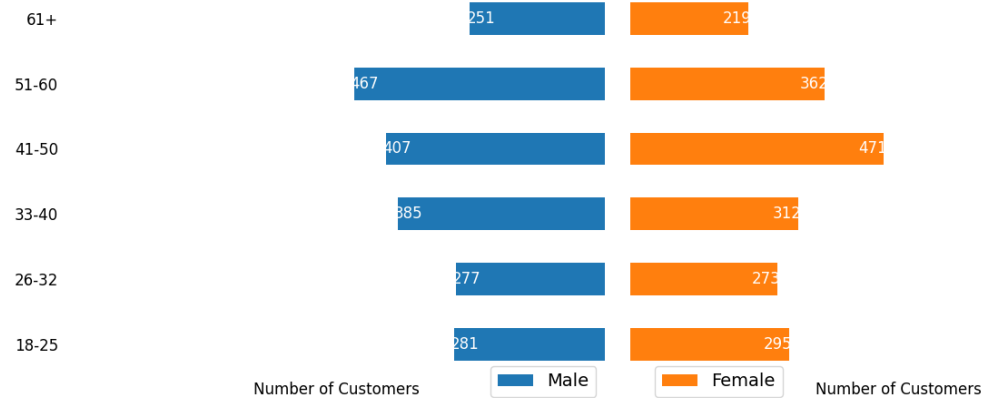
# Plotting Male counts
gender_age_group_counts["Male"].plot(kind="barh", ax=axes[0], color="tab:blue", xlim=[-1000, 0])
axes[0].set_ylabel("")
axes[0].set_yticklabels(gender_age_group_counts.index, fontsize=12)
axes[0].tick_params(left=False, bottom=False, labelbottom=False)
axes[0].set_xlabel("Number of Customers", fontsize=12)
axes[0].legend(["Male"], fontsize=14, bbox_to_anchor=(0., 0., 1., .05))
axes[0].set_frame_on(False)

# Plotting Female counts
gender_age_group_counts["Female"].plot(kind="barh", ax=axes[1], color="tab:orange", xlim=[0, 1000])
axes[1].set_ylabel("")
axes[1].set_yticklabels([]) # Hide y-axis labels as they are on the left side
axes[1].tick_params(left=False, bottom=False, labelbottom=False)
axes[1].set_xlabel("Number of Customers", fontsize=12)
axes[1].legend(["Female"], fontsize=14, bbox_to_anchor=(0., 0., 0.3, .05))
axes[1].set_frame_on(False)

# Annotate bars
for i, (male_count, female_count) in enumerate(zip(gender_age_group_counts["Male"], gender_age_group_counts["Female"])):
    axes[0].annotate(str(abs(int(male_count))), (male_count + 20, i), color="w", va="center", ha="center", fontsize=12)
    axes[1].annotate(str(int(female_count)), (female_count - 20, i), color="w", va="center", ha="center", fontsize=12)

plt.tight_layout(rect=(0,0,1,0.95))
plt.show()
```

### Perbandingan Jumlah Pelanggan Laki-laki dan Perempuan Berdasarkan Kelompok Usia

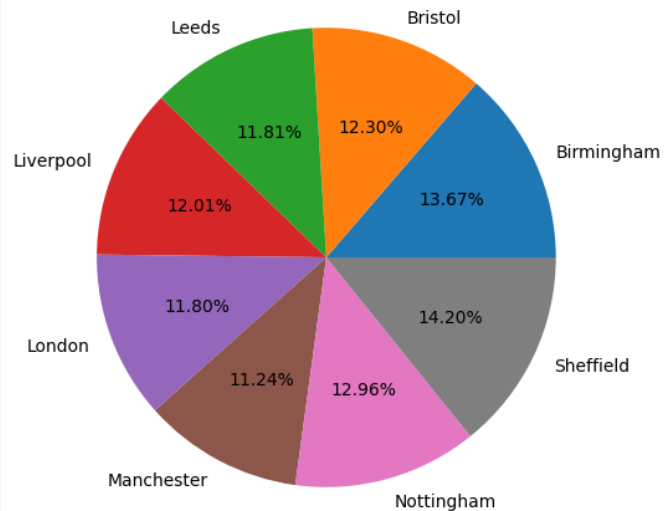


```

# Create a pie chart of distribution of cities where the customers come from
merged_df['gmv'] = merged_df['quantity'] * merged_df['price']
gmv_per_city_uk = merged_df.groupby('city')['gmv'].sum().reset_index()
plt.figure(figsize=(6,6))
plt.pie(gmv_per_city_uk['gmv'], labels = gmv_per_city_uk['city'], autopct='%1.2f%%')
plt.title('GMV Contribution Per City - UK in 2023 - 2024', loc='center', pad=30, fontsize=15, color='blue')
plt.show()

```

GMV Contribution Per City - UK in 2023 - 2024



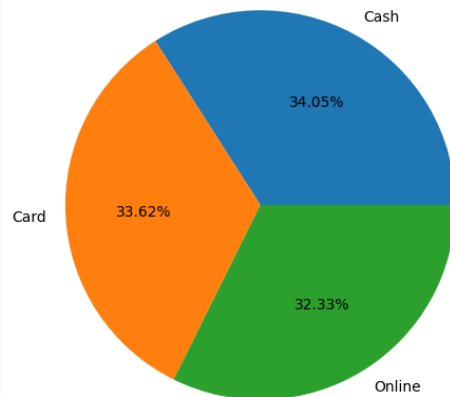
```

# Create a pie chart of distribution of payment methods
payment_method_counts = merged_df['payment_method'].value_counts().reset_index()
payment_method_counts.columns = ['payment_method', 'count'] # Rename columns for clarity
plt.figure(figsize=(6,6))
plt.pie(payment_method_counts['count'], labels = payment_method_counts['payment_method'], autopct='%1.2f%%')
plt.title('Payment Method Distribution', loc='center', pad=30, fontsize=15, color='blue')
plt.show()

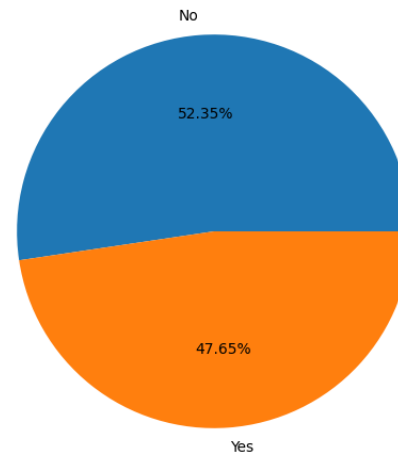
# Create a pie chart of distribution of loyalty member
loyalty_member_counts = merged_df['loyalty_member'].value_counts().reset_index()
loyalty_member_counts.columns = ['loyalty_member', 'count'] # Rename columns for clarity
plt.figure(figsize=(6,6))
plt.pie(loyalty_member_counts['count'], labels = loyalty_member_counts['loyalty_member'], autopct='%1.2f%%')
plt.title('Loyalty Member Distribution', loc='center', pad=30, fontsize=15, color='blue')
plt.show()

```

Payment Method Distribution



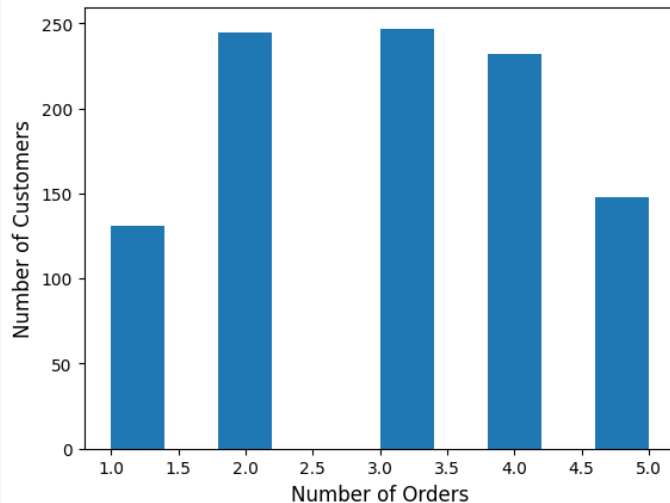
Loyalty Member Distribution



```
# Create a histogram for the distribution of number of orders per customer
data_per_customer = (merged_df.groupby('customer_id')
                      .agg({'order_id': 'nunique',
                           'quantity': 'sum',
                           'gmv': 'sum'})
                      .reset_index()
                      .rename(columns={'order_id': 'orders'}))

plt.clf()
plt.figure()
plt.hist(data_per_customer['orders'], range=(1,5))
plt.title('Distribution of Number of Orders per Customer\n UK 2023-2024', pad=30, fontsize=15, color='blue')
plt.xlabel('Number of Orders', fontsize = 12)
plt.ylabel('Number of Customers', fontsize = 12)
plt.show()
```

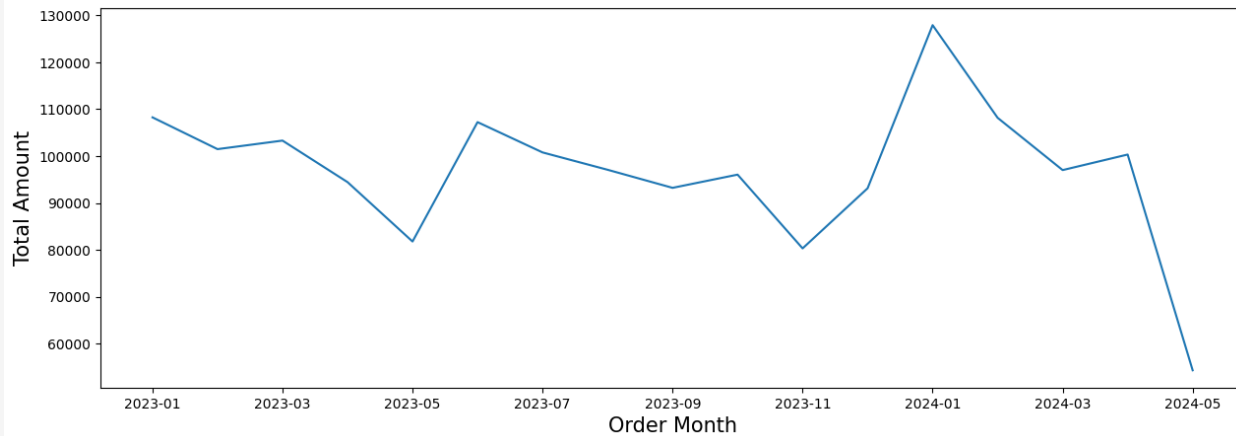
Distribution of Number of Orders per Customer  
UK 2023-2024



```
# Buat kolom order_month
import datetime
merged_df['order_date'] = pd.to_datetime(merged_df['order_date']) # Ensure order_date is datetime
merged_df['order_month'] = merged_df['order_date'].dt.strftime('%Y-%m')
# Buat kolom gmv
merged_df['gmv'] = merged_df['quantity'] * merged_df['price']

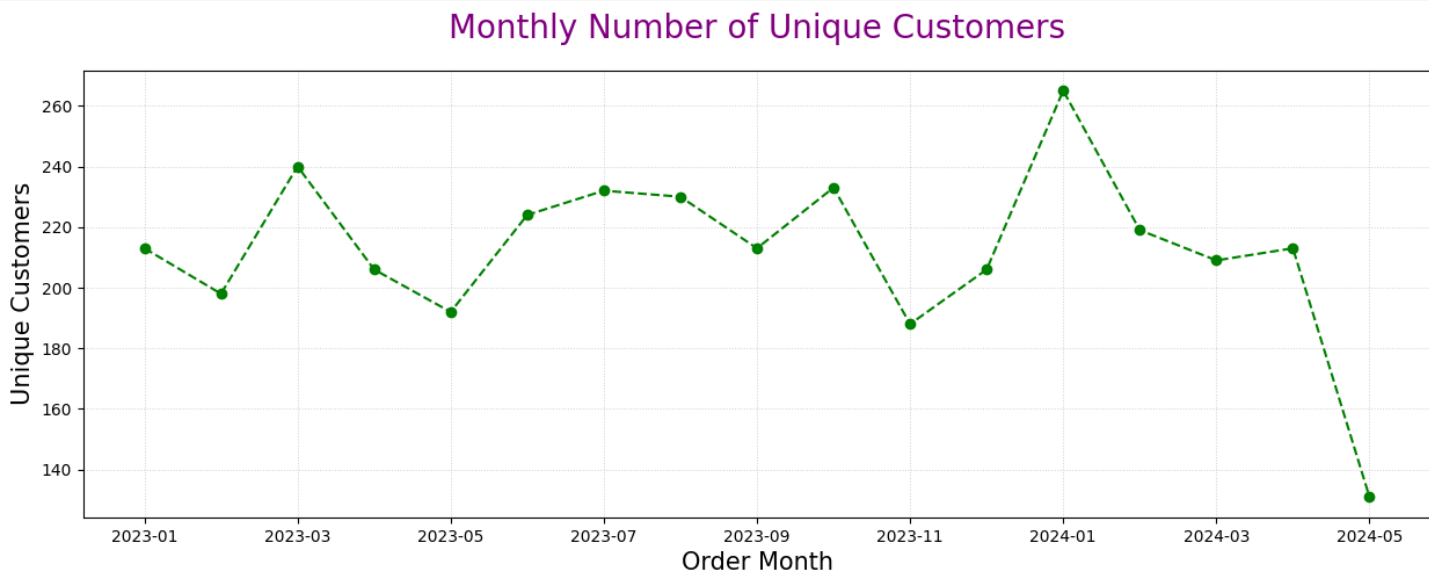
# Plot 1: Monthly GMV Year 2023
plt.figure(figsize=(15, 5))
merged_df.groupby(['order_month'])['gmv'].sum().plot()
plt.title('Monthly GMV Year 2023 - 2024', loc='center', pad=20, fontsize=20, color='blue')
plt.xlabel('Order Month', fontsize=15)
plt.ylabel('Total Amount', fontsize=15)
plt.show()
```

Monthly GMV Year 2023 - 2024

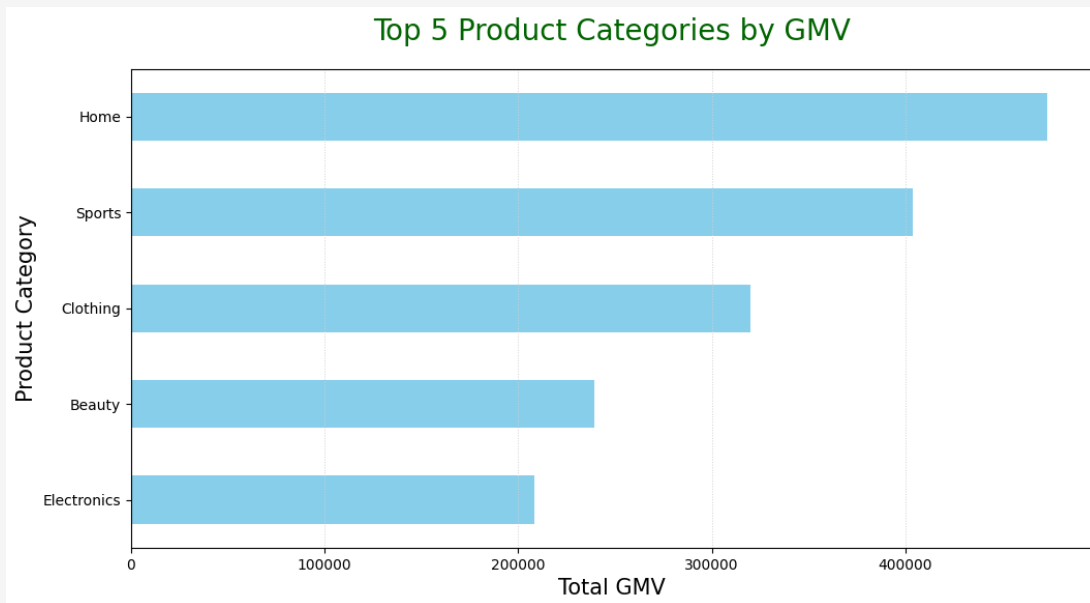




```
# Plot 2: Monthly Number of Unique Customers
plt.figure(figsize=(15, 5))
merged_df.groupby('order_month')['customer_id'].nunique().plot(color='green', marker='o', linestyle='--')
plt.title('Monthly Number of Unique Customers', loc='center', pad=20, fontsize=20, color='purple')
plt.xlabel('Order Month', fontsize=15)
plt.ylabel('Unique Customers', fontsize=15)
plt.grid(color='lightgray', linestyle=':', linewidth=0.7)
plt.show()
```

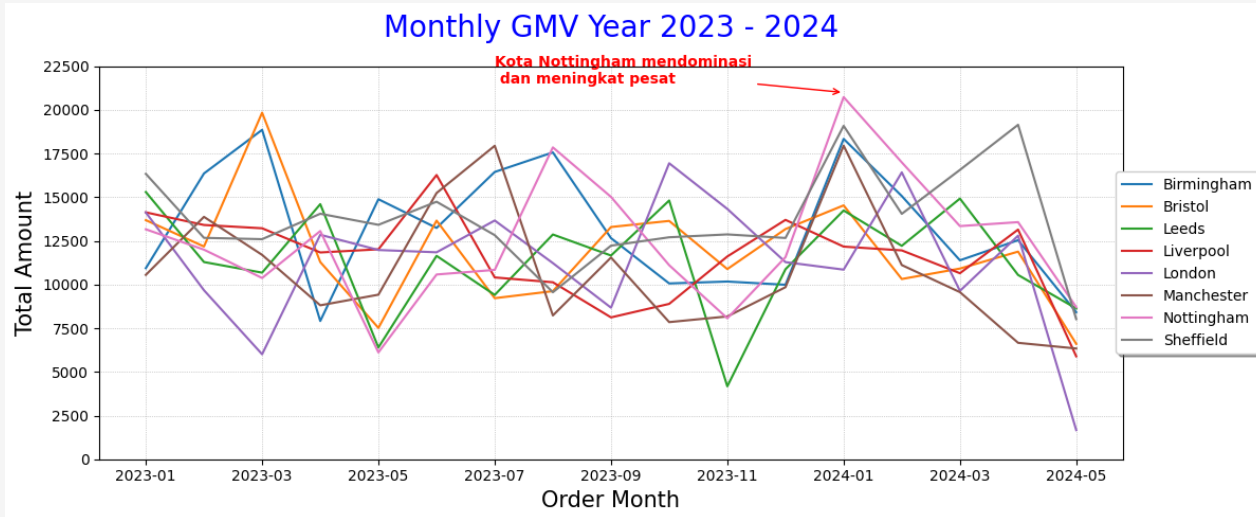


```
# Plot 3: Top 5 Product Categories by GMV
plt.figure(figsize=(12, 6))
top_categories_gmv = merged_df.groupby('category')['gmv'].sum().nlargest(5)
top_categories_gmv.plot(kind='barh', color='skyblue')
plt.title('Top 5 Product Categories by GMV', loc='center', pad=20, fontsize=20, color='darkgreen')
plt.xlabel('Total GMV', fontsize=15)
plt.ylabel('Product Category', fontsize=15)
plt.xticks(rotation=0)
plt.grid(axis='x', color='lightgray', linestyle=':', linewidth=0.7)
plt.gca().invert_yaxis() # To have the highest value on top
plt.show()
```



```
# Plot 4: Monthly GMV Year 2023 - 2024
plt.figure(figsize=(15, 5))
merged_df.groupby(['order_month', 'city'])['gmv'].sum().unstack().plot()
plt.title('Monthly GMV Year 2023 - 2024', loc='center', pad=20, fontsize=20, color='blue')
plt.xlabel('Order Month', fontsize=15)
plt.ylabel('Total Amount', fontsize=15)
plt.grid(color='darkgray', linestyle=':', linewidth=0.5)
plt.ylim(ymin=0)
labels, locations = plt.yticks()
plt.yticks(labels)
plt.legend(loc='right', bbox_to_anchor=(1.14, 0.5), shadow=True, ncol=1)

plt.annotate('Kota Nottingham mendominasi\ndan meningkat pesat', xy=(12, 21000), xytext=(6, 21500),
            weight='bold', color='red',
            arrowprops=dict(arrowstyle='->', connectionstyle='arc3', color='red'))
plt.gcf().set_size_inches(12, 5)
plt.tight_layout()
plt.show()
plt.show()
```



# Kesimpulan

1. Ada terdapat **1,200 customer/pelanggan** yang berbelanja di perusahaan retail ini selama 2023 sampai 2024. Mayoritas pelanggan berjenis kelamin **laki-laki** dan berasal dari **Kota Sheffield**. Rata-rata usia pelanggan adalah sekitar **41 tahun**, rata-rata barang yang dibeli adalah sebanyak **3 unit**, dan rata-rata pengeluaran setiap pelanggan adalah **£1,431.5**. Total pengeluaran terbanyak yang dikeluarkan oleh pelanggan adalah sebesar **£5,124.2**.
2. Mayoritas pelanggan menggunakan metode pembayaran **cash** dan **tidak memiliki loyalty membership**.
3. Produk terlarisnya adalah **Vacuum Cleaner 6 (P006)** dengan unit yang terjual sebanyak **177 unit**. Kategori produk yang paling banyak berkontribusi untuk pendapatan tersebut adalah kategori **Home**. Total pendapatan yang diperoleh perusahaan sejak Januari 2023 hingga Mei 2024 adalah **£1,644,817.7**. Pendapatan tersebut didominasi oleh penjualan pada Desember 2023 hingga Januari 2024, di mana saat itu mengalami lonjakan pelanggan yang berbelanja, sehingga pendapatan di cabang **Kota Nottingham** juga naik drastis selama bulan tersebut. Namun, total pelanggan dan pendapatan di sekitar April – Mei 2024 serentak menurun di seluruh cabang.

# Rekomendasi

1. Dikarenakan sebagian besar pelanggan tidak memiliki Loyalty Membership, maka hal tersebut perlu menjadi bahan evaluasi bagi tim sales perusahaan. Kemungkinan program Loyalty Membership tersebut kurang efektif.

**Terima kasih!**